Milestone 1 Report

Mastodon Scaling Study

CS6650 Distributed Systems — Spring 2026

Team: Yaoyi Wang & Yehe Yan

---

1. Problem, Team, and Overview of Experiments

Our project studies how a federated social network behaves under load by using Mastodon as a real-world distributed systems case study. We chose Mastodon because it is not just a web application: it combines asynchronous workers, database and cache coordination, and cross-instance federation through ActivityPub. This makes it a good system for studying bottlenecks, internal component dependencies, and distributed behavior across independently deployed instances.

Beyond its technical complexity, Mastodon embodies a fundamentally different philosophy of distribution. Unlike a centralized platform where a single company absorbs all load behind layers of engineered fault tolerance, Mastodon distributes responsibility — socially and physically — across independently operated nodes. Heavy requests are naturally spread among instances rather than funneled through one infrastructure owner. This means a single node going offline causes only localized disruption, and taking the network down entirely is structurally implausible. In this sense, Mastodon's resilience is not engineered into any one deployment; it emerges from decentralization itself.

This also makes Mastodon a useful pedagogical subject. A single instance contains all the essential components of a modern social platform — a relational database, an in-memory cache, asynchronous job queues, a web frontend, and a federation protocol — while remaining small enough to deploy, instrument, and stress-test in a student environment. It is, in effect, a textbook template for understanding how social network infrastructure is built and where it breaks.

This problem is important because future stakeholders who deploy social platforms, collaboration tools, or federated systems need to understand not only how a single service performs, but also how background jobs, caching, database coordination, and cross-instance communication affect performance and reliability. Mastodon gives us a realistic way to explore these tradeoffs.

Our team is organized around two complementary technical directions. Yaoyi focuses on the single-instance performance path, especially load generation, latency analysis, and bottleneck identification on Instance A. Yehe focuses on backend and dependency-oriented behavior on Instance B, including queue, cache, and database-related observations. Federation experiments and final integration are shared between both teammates.

At the current stage, our experiments are organized into three lines of investigation:

1. **Single-instance bottleneck** — identify which component becomes the bottleneck first under load.
2. **Bottleneck shifting / component dependency** — observe whether changing web-side capacity or internal constraints pushes pressure toward PostgreSQL, Redis, Sidekiq, or connection limits.

3. **Federation validation / propagation behavior** — validate whether two independently deployed Mastodon instances can support a basic federated workflow and observe simple propagation or recovery behavior across instances.

We used AI as a support tool throughout the project, mainly for deployment debugging ideas, experiment framing, and documentation refinement. The benefit was speed: AI helped us quickly generate alternative deployment paths, compare experiment options, and organize our observations into a clearer distributed-systems story.

However, AI suggestions were not always reliable in our actual environment. In several cases, proposed solutions assumed a more permissive cloud setup than AWS Academy Learner Lab actually provides. Because of this, all AI-assisted suggestions had to be manually validated through commands, logs, screenshots, and real deployment outcomes.

To support observability, we rely on lightweight but practical tools: Locust statistics and charts, Docker stats, container logs, curl-based health checks, and Mastodon UI screenshots. Since our environment is AWS Academy Learner Lab, this lightweight observability approach is more practical than a heavy production monitoring stack.

---

2. Project Plan and Recent Progress

Our original proposal planned to deploy Mastodon using a production-style CloudFormation / ECS / Fargate architecture. During the first phase of the project, we attempted multiple simplifications of the `widdix/mastodon-on-aws` path, including removing SES, CloudFront, S3, Route 53 / ACM-managed components, and VPC Flow Logs. Despite these changes, the deployment still failed because AWS Academy Learner Lab blocked required IAM resources, especially ECS task-related role creation.

Because of this, our recent progress has centered on a deployment pivot. Instead of continuing to pursue the original cloud-native path, both teammates pivoted to a simplified EC2 + Docker Compose architecture. This allowed us to recover the project and begin collecting meaningful performance and federation results.

Recent progress includes the following:

- validated that the original CloudFormation / ECS deployment path was infeasible in Learner Lab;
- deployed two working Mastodon instances on EC2 using Docker Compose;
- configured Nginx + HTTPS with Let's Encrypt for both instances;
- completed initial single-instance load testing on Yaoyi's instance;
- completed authenticated and rate-limit-related observations on Yehe's instance;
- validated basic federation between the two instances.

Our current task breakdown is:

- **Yaoyi:** single-instance bottleneck analysis, worker / bottleneck-shifting observations, result formatting, and screenshots.
- **Yehe:** component dependency experiments, including queue, cache, and database-connection-related observations.
- **Shared:** federation validation, possible failure / recovery test, final report integration, and presentation.

Our current timeline is:

- **Week 1:** deployment feasibility investigation and pivot decision.
- **Week 2:** EC2-based deployment stabilization and initial experiments.
- **Week 3:** component dependency experiments and federation follow-up tests.
- **Week 4:** final analysis, report writing, and presentation preparation.

AI played a practical but limited role in our planning process. On the positive side, it helped us iterate faster when the original CloudFormation / ECS deployment path failed. It was useful for generating debugging hypotheses, proposing fallback deployment options, and helping us restructure the project after the pivot to EC2 + Docker Compose.

At the same time, AI also had clear limitations. Some suggestions were too generic, too optimistic, or not aligned with the restrictions of AWS Academy Learner Lab. In other words, AI was useful for brainstorming and reframing the project, but not trustworthy enough to replace direct technical validation. This became an important lesson in itself: AI can accelerate engineering work, but it cannot substitute for systems understanding, manual debugging, and environment-specific verification.

---

3. Objectives

Our short-term objective is to complete a meaningful distributed systems case study using Mastodon despite the deployment restrictions imposed by AWS Academy Learner Lab. Concretely, this means completing a strong single-instance bottleneck analysis, extending the work with at least one component-dependency experiment, and validating cross-instance federation behavior with clear evidence.

Our longer-term objective is to understand Mastodon not only as a web application, but as a distributed system whose behavior depends on multiple interacting layers: web workers, background jobs, Redis, PostgreSQL, reverse proxy configuration, and ActivityPub-based federation. Beyond the course, this project could be extended into a more realistic study of federated-system latency, asynchronous delivery, retry behavior, queue backlog growth, or failure recovery under broader multi-instance activity.

Observability is central to both the short-term and long-term goals. For this project, we aim to support observability in a concrete and realistic way using metrics and traces that are feasible in our environment: Locust charts, Docker resource snapshots, queue and log observations, and end-user federation visibility checks.

AI is not part of the deployed Mastodon system itself, but it is part of our development workflow. To ensure reliability and cost control, we treat AI outputs as suggestions only. We do not trust AI-generated instructions without validating them through actual deployment, testing, or log inspection.

A secondary objective of this project is to develop stronger engineering judgment in an AI-assisted workflow. Beyond completing the Mastodon case study itself, we want to show that we can use AI productively without over-trusting it: generating ideas quickly, validating them carefully, and adapting when real systems constraints contradict AI-generated suggestions.

---

4. Related Work

This project connects directly to several core distributed systems themes from the course: bottleneck identification, queueing and asynchronous work, failure and recovery, and cross-instance consistency behavior. Mastodon is a strong case study because it combines local service interactions with a distributed federation layer, making it more representative than a single centralized monolith.

Outside the course, Mastodon and ActivityPub documentation provide a useful background for understanding how posts, follows, and interactions propagate between independent servers. We also draw on practical system knowledge around Redis, Sidekiq, PostgreSQL, and reverse proxies in distributed web applications.

We identified three related projects from the class:

Distributed Music Streaming Simulation (Ananyaa Mylsamy & Manasha Sekar) studies bottlenecks, concurrency limits, and horizontal scaling behavior using AWS ECS Fargate and Locust. This is similar to our project because both focus on identifying where performance breaks down under load and whether the system scales. The main difference is that their project is built around music streaming and QoS-specific metrics like rebuffering latency and adaptive bitrate switching, while ours focuses on Mastodon federation and cross-instance social interactions.

Pattern Learning & Crowd-Validated Content Generation (Sam Wu & Jingsi Zhang) studies distributed backpressure, checkpoint-based failure recovery, and cache concurrency safety in an LLM processing pipeline. This is similar to our project because both study how distributed systems behave under load and partial failure — their backpressure and checkpoint recovery experiments parallel our interest in bottleneck identification and queue behavior. The main difference is that their system is an AI content pipeline with explicit fanout stages, while ours is a federated social network where background jobs and cross-instance delivery are the key distributed behaviors.

Letterboxd for Video Games (Nicholas Soares) is a microservice backend that studies cascading failure resilience, consistency under heavy write load, and Redis caching tradeoffs over PostgreSQL. This is similar to our project because both use a Redis + PostgreSQL stack and study system behavior under load. The main difference is that his project focuses on microservice coordination and consistency within a single deployment, while ours focuses on federation across independently deployed Mastodon instances communicating through ActivityPub.

These related projects help position our work within the broader set of distributed systems studies in the class. Our project is distinctive because it combines real deployment constraints, single-instance stress behavior, and cross-instance federation — all in a system that was not purpose-built for benchmarking, but is a real open-source application.

---

5. Methodology

Our methodology is based on a simplified but functional Mastodon deployment using EC2 + Docker Compose. Each teammate runs one Mastodon instance, and the two instances are connected through valid domains and HTTPS. This allows us to perform both local load-testing experiments and cross-instance federation tests in a constrained but realistic environment.

Experiment 1 — Single-Instance Bottleneck

For the first experiment, we use Locust to generate read-heavy traffic against Yaoyi's Mastodon instance. We measure throughput, response latency, error rate, and container-level resource usage using Locust charts and Docker stats. The goal is to identify which component becomes the first bottleneck under increasing load. This experiment establishes a baseline before we attempt more detailed dependency-oriented tests.

Experiment 2 — Bottleneck Shifting / Component Dependency

For the second experiment, we modify one system factor at a time to see whether the observed bottleneck moves deeper into the stack. Candidate factors include web-side worker capacity, rate-limit behavior, Redis/cache effects, Sidekiq queue behavior, and database connection constraints. The purpose is not to claim production-grade autoscaling, but to study whether the application layer, cache layer, queue layer, and DB layer interact in ways that reveal new bottlenecks or system sensitivities.

Experiment 3 — Federation Validation / Simple Failure Test

For the third experiment, we validate federation between the two Mastodon instances. At minimum, this includes remote account discovery, mutual follow, post visibility, and remote interaction propagation. If time allows, we will also run a small follow-up test involving simple latency observation or a partial-outage scenario in which one instance is temporarily unavailable and we observe whether the other catches up after restart.

AI is used in this methodology only as a support tool for planning and debugging. It helps us generate deployment alternatives, experiment ideas, and clearer documentation, but all commands, results, and conclusions are manually validated in our real environment.

Observability is supported by:

- Locust charts and statistics;
- Docker stats and container logs;
- Sidekiq monitor dashboard;
- curl-based health and endpoint checks;
- Mastodon UI screenshots showing interaction or propagation outcomes.

The main tradeoffs we evaluate are:

- local component bottlenecks versus deeper dependency bottlenecks;
- simplicity of tiny deployment versus fidelity to production;
- local responsiveness versus asynchronous / federated delivery behavior.

---

6. Preliminary Results

We already have preliminary results for two major parts of the project.

Experiment 1 — Single-Instance Bottleneck

On Yaoyi's instance, initial read-heavy Locust tests completed successfully with zero observed failures up to 500 users. The collected Locust charts and Docker observations suggest that the web layer is the first visible bottleneck, rather than PostgreSQL or Redis. This is an important early result because it indicates

that Mastodon's application layer and request-handling path may saturate before lower layers such as the database do under the current workload.

Component / Dependency Observations

On Yehe's instance, authenticated tests encountered rate-limiter behavior at relatively low concurrency. This suggests that application-level protections may prevent the system from reaching true hardware-level saturation in some scenarios. This also motivates the next experiment phase: if these protections are relaxed or if web-side capacity is adjusted, will the bottleneck shift toward PostgreSQL, Redis, Sidekiq, or connection limits?

Federation Preliminary Results

We have also completed a basic federation validation. The following interactions succeeded between the two independently deployed instances:

- remote account discovery,
- mutual follow,
- remote post visibility,
- remote like / favorite notification.

These results show that our EC2 + Docker Compose deployment is not only locally functional, but also capable of participating in a basic federated workflow.

What remains to be collected

To finish the milestone into a stronger final project, we still need:

- one bottleneck-shifting or component-dependency experiment with clearer before/after comparison;
- additional observations on queue behavior, cache effects, or DB connection constraints;
- one small federation propagation or partial-outage check if time allows.

Base-case and worst-case workload

Our current base-case workload is anonymous read-heavy browsing, because it is stable and easy to reproduce. Our expected worst-case workload is a more authenticated or write-heavy interaction mix, because it stresses not only the web layer but also asynchronous work, Redis, PostgreSQL, and possibly federation delivery paths.

---

7. Impact

We believe this project is significant because it studies a real federated social network rather than a toy benchmark. Mastodon is especially interesting for distributed systems because it combines local service performance with asynchronous background jobs, cache/database interaction, and federation across independent instances.

People should care about our results for two reasons. First, the project shows how environment constraints can force a major architecture pivot, which is a realistic systems engineering problem. Second, the project

demonstrates that distributed systems behavior is not just about adding more cloud resources: it is also about bottlenecks, component dependency, queueing, retry behavior, and cross-instance communication.

Other students in the class could potentially use our project in two ways. They could reference our lightweight EC2 + Docker Compose deployment approach as a reproducible path for constrained environments, and they could also interact with the live instances to help validate federation behavior or user-facing system observations. This makes the project both relevant and extendable.

This project is also useful as evidence of what we can contribute in an AI-assisted engineering environment. Rather than simply asking AI for answers, we used it to accelerate idea generation, then applied human judgment to validate, reject, or refine those suggestions. That ability to combine AI assistance with real systems reasoning is itself part of the project's value.